

CSWP: march on in time

Alichan Borokov Sam Hemati
`Alichan.Borokov@UGent.be` `Sam.Hemati@UGent.be`

Joran Van Haelst
`Joran.VanHaelst@UGent.be`

December 2025

1 Summary of the problem

In this assignment we implement the march-on in time algorithm on PEC scatterers. We implemented on TM case of the wave equation. This means that the electric field only contains a z-component and the magnetic field only in the x and y- direction. the problem is also consistent in the z-direction this means the material is symmetric under the z-axis. This reduces our problem to a 2 dimensional problem.

After that we use the greens function in its retarded form to solve the boundary integral equation which has as the unknown the current. At last we discretize the integral equation as done in the assignment. And we use those formulae to describe the different problems.

2 assignment

we validate our code by calculating the time domain scattering at a cylinder.

The formula description is done in the assignment. Our task is implement the march-on-in-time algorithm on this example and look if this corresponds with the analytical solution.

for the first part we calculate the timedomain scattering at a cylinder. Using an incoming wave going to the positive x direction.

3 Implementation

The algorithm is implemented in Python, using the Numpy and Scipy packages.

The MOT algorithm is fully contained in the MOT class, initiating an instance takes a few arguments:

param	value	unit	description
R	10	m	radius of PEC
t_0	9e-8	s	center of incident pulse
T	20	m	width of incident pulse
dt	1e-9	s	timestep
t_{end}	64e-8	s	end of simulation
N_S	32	1	number of segments for PEC
N_G	8	1	order of Gaussian quadrature

Table 1: Parameters for validation

- **curve** is a **numpy** array of shape $(2, N_S + 1)$. The first index is the coordinate (0 for x or 1 for y , generally `_x` and `_y` were written as aliases for 0 and 1). N_S is the number of segments in which the curve has been discretized. This field consists out of the edges of these segments (the first and last elements must equal),
- **dt** is the time step in seconds,
- **N_T** is the number of timesteps taken, and
- **N_G** is the number of points for the Gaussian quadrature.

With these arguments the quadrature points (**coordquad** and **weights**), the geometry fields (**rho** are the midpoints, **tangents** the tangent vectors and **l** the lengths of the segments, **rhop**[**_x**, **m**, **g**] is the x -coordinate of the m -th segment, linearly interpolated for x_g), F (an array of shape (N_T, N_S, N_S, N_G) , such that $F(k, \rho_m, \rho_{n,g}) = F[\mathbf{k}, \mathbf{m}, \mathbf{n}, \mathbf{g}]$) and Z (with shape (N_T, N_S, N_S) , so that $(Z_k)_{m,n} = Z[\mathbf{k}, \mathbf{m}, \mathbf{n}]$) are calculated.

Finally the function **solve** implements the stepping algorithm. It takes one argument **V**, which is an array of shape (N_T, N_S) , such that $(V_j)_m = E^i(\rho_m, j\Delta t) = V[\mathbf{j}, \mathbf{m}]$. The solver (and **_init.Z**) uses the Numpy function **einsum**, which makes translating sums from math into code a lot easier:

$$\sum_g^{N_G} w_g F(k, \rho_m, \rho_{n,g}) = \text{np.einsum}('g, kmng \rightarrow kmn', \text{weights}, F)$$

This and a few reshapes which had to happen for Numpy to be able to correctly broadcast together some arrays, are the only noteworthy difference between the code and its mathematic description from the assignment.

TODO maybe some stuff about the second exercise and the E^i calculation.

4 Validation

For the validation the MOT-algorithm is ran with as incident field a pulse. The incident wave is plotted in figure 1 at the ‘sun’- and shadow-side. The

parameters T and t_0 are chosen in such a manner that the incident field is zero at $t = 0$ over the whole PEC and such that the width in the frequency domain is wide enough to cover the range $\omega/c \in [0, 1]$. Their values are given in table 1.

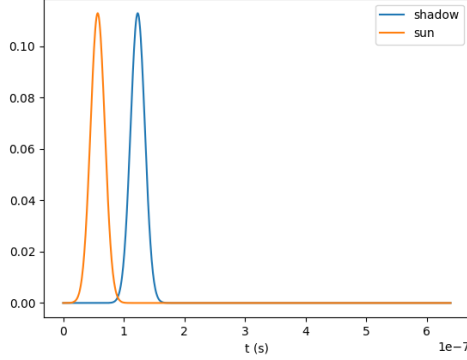


Figure 1: Incident field at the sun- and shadow-side

The solution of the MOT-algorithm can easily be integrated to obtain the current induced on the PEC. It is shown in figure 2 in the form of a polar plot and in figure 3 as a function of time.

The spectrum of the incoming wave quickly decays to zero. This means that when dividing by this spectrum the result will explode. Due to this the frequency range for which the solver can be used is only limited. We chose to limit the frequency range to the region where the spectrum of the incoming wave is more than 0.1 % of its maximum.

When calculating the Fourier transform of the solution from the MOT-algorithm, Numpy's `rfft` function was used. It is important to note that this is a discrete transform without any normalization factor. To compare it with the continuous transform, a correction factor of $dt/\sqrt{2\pi}$ is needed.

4.1 Analytical Solution

$$e_z(\vec{\rho}) = \sum_{n=-\infty}^{\infty} j^n \left(J_n(k\rho) - \frac{J_n(ka)}{H_n^{(2)}(ka)} H_n^{(2)}(k\rho) \right) e^{jn\phi} \quad (1)$$

The analytical solution (1) for e_z is copied from appendix B from the assignment. The derivate towards ρ is taken and evaluated at $\rho = a$:

$$\sum_{n=-\infty}^{\infty} j^n \left(k J'_n(ka) - k \frac{J_n(ka)}{H_n^{(2)}(ka)} H_n'^{(2)}(ka) \right) e^{jn\phi}.$$

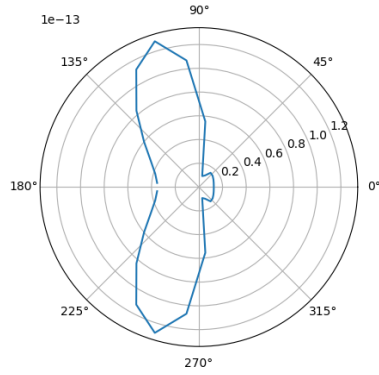


Figure 2: Polar plot of the current at $t = t_0$

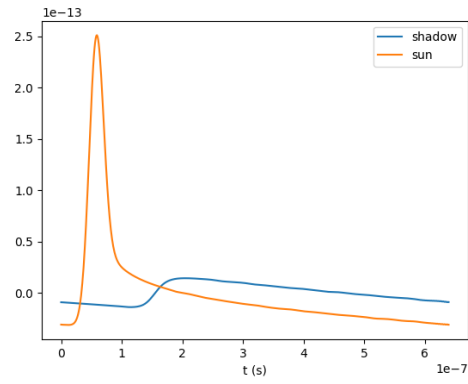


Figure 3: Plot of the current at the sun- and shadow-side

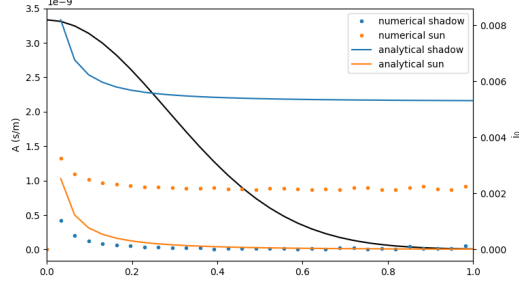


Figure 4: Current at the sun-side and shadow-side calculated numerically and analytically

With a bit of refactoring,

$$\sum_{n=-\infty}^{\infty} \frac{j^n k}{H_n^{(2)}(ka)} \left(J'_n(ka) H_n^{(2)}(ka) - J_n(ka) H_n'^{(2)}(ka) \right) e^{jn\phi},$$

the Wronskian relationship is revealed, so that finally equation 2 follows.

$$j_z = \frac{1}{j\omega\mu} \sum_{n=-\infty}^{\infty} \frac{j^n k}{H_n^{(2)}(ka)} \frac{2j}{\pi ka} e^{jn\phi} \quad (2)$$

The summation in this equation was cut off at $ka + 2 \approx N \in \mathbb{Z}$, which was established by the rule of thumb that $H_n^{(2)}(z)$ decays exponentially as soon as $n > z$ in combination with some experimentation.

4.2 Discussion

In figure 4 the numerically and analytically computed current in the frequency domain are plotted, alongside with the spectrum of the incoming wave (in black). The analytical and numerical current follow the same shape, but don't match at all. Even stranger is that the shadow-side from the analytical solution seems to lie higher than its sun-side counterpart. ...

The width of the incoming pulse can be made smaller, so that in the frequency domain it becomes wider. This cannot be continued indefinitely, as the timestep dt becomes too small to represent the pulse correctly in the time-domain. Making dt smaller makes that the pulse more correctly approximates a Gaussian pulse, but also makes the maximum frequency (after the Fourier-transform) bigger. At these maximum frequencies the spectrum of the incoming pulse is again nearly zero, so that the additional range in frequencies gained is useless. Thus a tradeoff between the width of the pulse and the timestep must be made. Another parameter that can be adjusted is t_{end} . As this increases the resolution in the frequency domain also increases, so that finer details are

visible. Increasing this (or decreasing dt) comes at a computational cost, as these increase the total steps in time N_T , so that the computation takes longer. Increasing the radius of the PEC increases the difference between the current at the shadow-side and at the ‘sun’-side. The same effect can also be accomplished by decreasing the width of the incoming pulse.

Lastly N_S and N_G can be changed. The number of segments must be great enough so that the geometry of the PEC is correctly approximated. Increasing this any further does not increase the accuracy very much. Increasing N_S also comes at the cost of requiring more memory. The order of the Gaussian quadrature does not have much influence. As, once the geometry is well approximated, ρ_m are close together, so that $|\rho_m - \rho'|$ in F is small and can easily be approximated by a low order polynomial.

4.3 Reproduction

All plots (along with some extra animations) in the validation section can be reproduced by executing the function `Q_6.1_validation` near the end of the file, with as arguments the parameters given by table 1.

5 Resonance Frequencies of Cylindrical Cavity

5.1 Incident Field by Numerical Quadrature

5.2 Resonance Modes

5.3 Spectrum Numerical Solution

5.4 Off-center Source

6 Creative Assignment